

MSF Eastern Africa Online Donations Portal

Submission answer sheet for the Full Stack Web Developer written test.

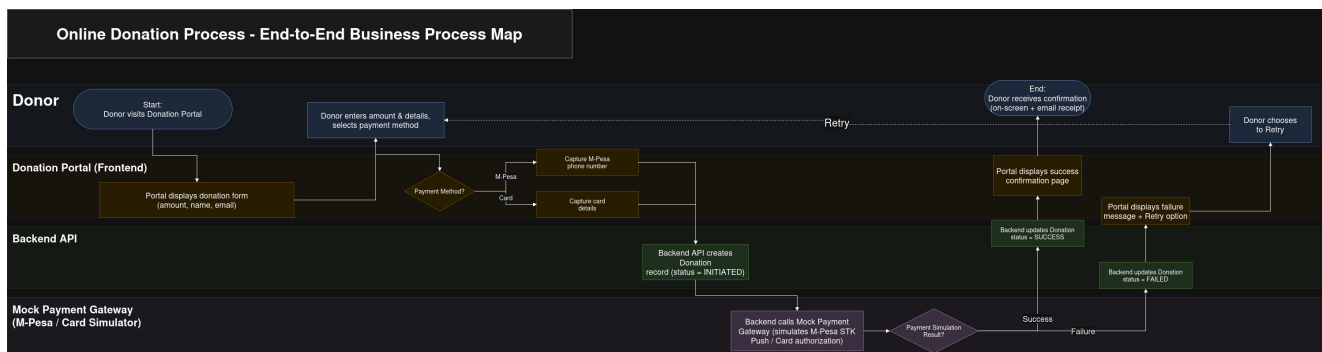
This document is intentionally anonymous: it contains no personally identifying information, in compliance with the test instructions. By submitting, I affirm that the work is my own and free of plagiarism.

Deliverables

Deliverable	Location
GitHub repository	Open repository on GitHub
Hosted application (frontend)	https://donation-app.stinkerer.workers.dev/
Hosted backend API	https://donation-demo-app.maunode.com/api/
API documentation (Swagger UI)	https://donation-demo-app.maunode.com/api/docs/
Business process map	Embedded below
Wireframes	Embedded below

1. Business Process Map

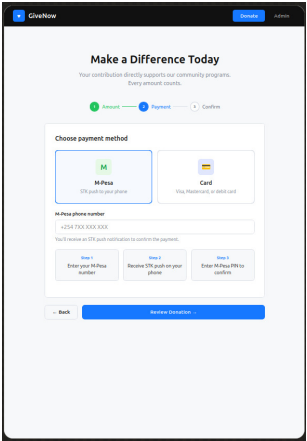
The diagram below illustrates the end-to-end donation process, from donation submission through payment confirmation, including the retry path for failed payments.



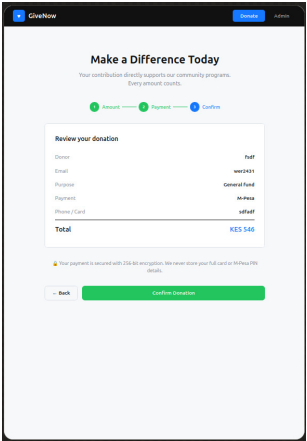
Business process map: donation submission to payment confirmation.

2. Wireframe

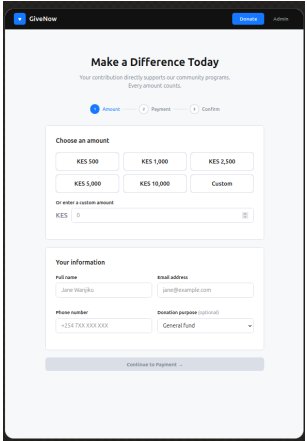
The wireframes below illustrate the proposed donation page user interface across the donation flow: amount entry, payment-method selection, M-Pesa and Card input states, review and confirm, and the success confirmation. The implemented UI realizes these as a 3-step wizard (Amount, then Payment, then Review and Confirm) followed by a receipt screen, with a live donations list and summary statistics alongside.



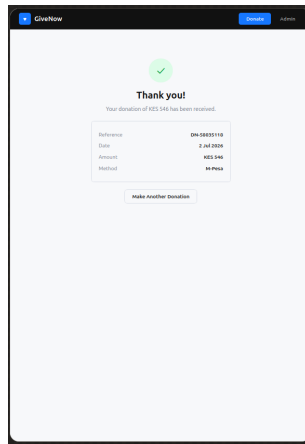
Wireframe 1



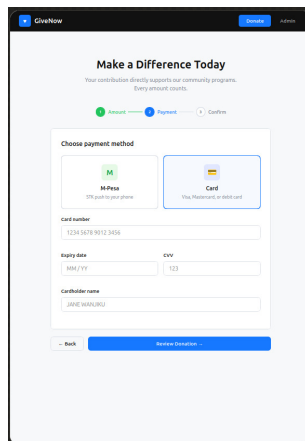
Wireframe 2



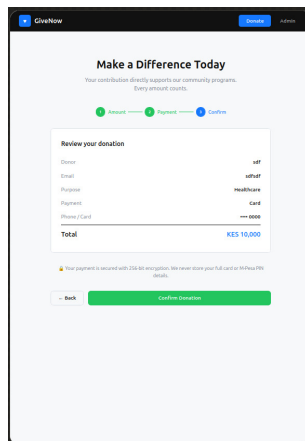
Wireframe 3



Wireframe 4



Wireframe 5



Wireframe 6

3. Solution Overview

A responsive one-page donations application:

- The donor enters their name, email, and donation amount, then selects M-Pesa or Card (with method-specific fields: M-Pesa phone number, or card number and expiry).
- A review and confirm step shows the full summary before submission.
- On confirm, the backend validates the request, creates a donation record (status = initiated), runs the mock payment gateway, and returns the final success or failed status with a transaction id and message.
- The UI shows loading (spinner during processing), success (receipt with transaction reference), or failure (gateway message and a Retry payment button that calls the retry endpoint).

- A security assurance banner (256-bit encryption; we never store your full card or M-Pesa PIN) is shown on the review step to build donor trust.
- A donations list with summary stats (total, successful, amount raised, currencies) provides transparency and updates after each donation.

The backend API is RESTful, returns appropriate HTTP status codes (201 on success, 200 on a processed-but-failed payment, 400 on validation errors), and is documented with OpenAPI/Swagger via drf-spectacular.

4. Technology Stack and Rationale

Layer	Technology
Backend	Django REST Framework and SQLite
Packaging	uv (Python dependency management)
Frontend	React 19, TypeScript, Vite, Tailwind CSS v4
Tests	pytest and pytest-django (backend); vitest and React Testing Library (frontend)
API docs	drf-spectacular (OpenAPI 3 and Swagger UI)
Deploy	Backend: Docker on a VPS. Frontend: Cloudflare Workers (static assets)

Why these frameworks

- Django REST Framework: first-class serializers, validation, viewsets, and auto-generated OpenAPI docs (drf-spectacular) keep the API small and self-documenting.
- React, TypeScript and Vite: a typed, component-based SPA gives a smooth wizard UX with compile-time safety; Vite keeps the dev loop fast and emits a static bundle.
- Tailwind CSS v4: utility-first styling keeps the UI responsive and consistent; responsive variants (sm:, lg:) make mobile-first layouts trivial.
- pytest and vitest: focused, fast unit and integration tests on both sides.

Assumptions

- No real payment provider is integrated; the gateway is mocked with a 60% success rate (the Stripe test card 4242 4242 4242 4242 always succeeds for deterministic testing).
- Full card numbers are never persisted; only the last 4 digits are stored.
- SQLite is used as the demo database and committed as a project artifact so the deployed demo starts with migrations applied and a superuser ready.
- The donation currency is KES by default in the UI; the model supports any 3-letter code.
- Donations are public (visible in the list) but donor card details are not.

How I would secure the application in production

- HTTPS only: enforce TLS on both origins; redirect HTTP to HTTPS.
- Secrets: move SECRET_KEY, DB credentials, and provider keys to environment variables or a secrets manager; never commit them.
- Set DEBUG = False, restrict ALLOWED_HOSTS to the real domains, and replace CORS_ALLOW_ALL_ORIGINS with an explicit allow-list of the frontend origin.
- Payments: integrate a real, PCI-compliant provider (Safaricom Daraja for M-Pesa; Stripe/PayPal for cards). Tokenize cards on the provider side so full PANs never touch our servers. Use signed webhooks for async payment confirmation.
- Rate limiting and CSRF: per-IP throttling on the donation endpoint; CSRF protection on mutations.

- Auth: authentication and authorization for any admin or reporting routes; the public donation endpoint stays open but throttled.
- Observability: structured logging and error tracking (e.g. Sentry); audit trails for payment status changes.
- Database: move from SQLite to PostgreSQL for concurrent-write safety.

Future improvements

- Webhook-based, asynchronous payment confirmation instead of a blocking call.
- Donor email receipts (success/failure) and a donation-history view by email.
- Recurring donations and donation goals or campaigns.
- Internationalization and multi-currency conversion.
- Pagination or infinite scroll on the donations list and richer summary charts.
- A CI/CD pipeline running both test suites on every push.

5. API Documentation

Interactive docs (Swagger UI): <https://donation-demo-app.maunode.com/api/docs/>.

Method	Endpoint and description
POST	/api/donations/: create a donation and process payment
GET	/api/donations/: list donations (paginated, ordered)
GET	/api/donations/{id}/: retrieve a single donation
POST	/api/donations/{id}/retry/: retry a failed payment
GET	/api/donations/summary/: aggregate stats by currency and method

Payment flow

- Donor submits details; the backend creates a record with payment_status = initiated.
- Mock gateway simulates M-Pesa STK Push or card authorization (60% success; test card 4242 4242 4242 4242 always succeeds).
- Backend updates the record to success or failed with a transaction id and message, and returns the donation.
- Frontend shows a success receipt (with transaction reference) or a failure screen with a Retry payment button (which calls the retry endpoint).
- Full card numbers are never stored; only the last 4 digits are kept.

6. Running and Tests

Backend (Django REST Framework):

```
cd backend && uv sync && uv run python manage.py migrate && uv run python manage.py runserver 0.0.0.0:8000
```

Frontend (React and Vite):

```
cd frontend && npm install && npm run dev (http://localhost:5173, proxies /api to :8000)
```

Tests:

- Backend: `cd backend && uv run pytest -v`, 28 tests.
- Frontend: `cd frontend && npm test`, 21 tests.

- Coverage: model behavior, API creation for M-Pesa and Card, validation (missing phone, invalid or expired card, invalid expiry format), the retry endpoint, summary aggregation, payment-gateway simulation, the wizard flow, payment-failure and retry UI, and the donations list states.